



MULTI-MODAL SUPERMARKET PRODUCT MATCHING WITH MACHINE LEARNING

(8,462 WORDS)

KA WAI KO

THESIS SUBMITTED IN PARTIAL FULFILLMENT
OF THE REQUIREMENTS FOR THE DEGREE OF
MASTER OF SCIENCE IN DATA SCIENCE & SOCIETY
AT THE SCHOOL OF HUMANITIES AND DIGITAL SCIENCES
OF TILBURG UNIVERSITY

STUDENT NUMBER

421371

COMMITTEE

dr. Dimitar Shterionov
dr. Raquel Garrido Alhama

LOCATION

Tilburg University
School of Humanities and Digital Sciences
Department of Cognitive Science &
Artificial Intelligence
Tilburg, The Netherlands

DATE

December 2, 2021

ACKNOWLEDGMENTS

Special thanks to Jumbo Supermarket that supplies the data necessary for this thesis project.

MULTI-MODAL SUPERMARKET PRODUCT MATCHING WITH MACHINE LEARNING

(8,462 WORDS)

KA WAI KO

Abstract

This paper examines various ways to improve supermarket product matching as a two-step process combining embedding generation and classification. For embedding generation, results show that transfer learning with triplet loss and multi-modal modeling can enhance embedding quality in terms of top-1 accuracy compared to pre-trained models, even when the training data consists of supermarket products from as many as 15 main categories. As for classification, multi-label classification outperforms binary classification in detecting matches with higher exact match ratio and precision score, showing its ability to reduce false positives and give more accurate matching results. The research also finds that adding size-related features to the multi-label classifier can further enhance the exact match ratio by as much as 4% while private label indicators only slightly improve the chosen evaluation metrics with the given data. A revised two-step approach is therefore proposed for one-way matching of competitor products with a supermarket's own products. The first step is generating embeddings with the multi-lingual Universal Sentence Encoder, DenseNet-201 and a multi-modal model all fine-tuned with triplet loss, and the second step is multi-label classification with cosine embedding distances and size features as input to determine which of the ten closest own products should be matched with a given competitor product.

1 DATA SOURCE / CODE / ETHICS STATEMENT

Work on this thesis did not involve collecting data from human participants or animals. The original owner of the data used in this thesis retains ownership of the data during and after the completion of this thesis. The author of this thesis acknowledges that they do not have any legal claim to this data. The code used in this thesis is only accessible by reviewers due to confidentiality requirements by the external thesis partner.

2 PREFACE

Thank you for taking the time in reading this thesis on supermarket product matching, a topic that is somewhat closely related to our daily lives. I would like to express my gratitude to my internal supervisor dr. Dimitar Shterionov for his professional guidance, and external supervisor Mr. Wendell Kuling for sharing his experience in the data science field with me. This thesis project would not have been completed without their support. I hope you enjoy reading this thesis and hopefully also get something out of it.

3 INTRODUCTION

The aim of this research project in collaboration with Jumbo Supermarket is to propose an effective workflow for automated product matching in the context of a supermarket. To achieve this goal, we will review existing methods of product matching and examine various ways that can potentially improve their performance on supermarket products.

Product matching refers to the process of identifying identical or intrinsically similar products, and it plays a key role in pricing strategy formulation of all supermarkets. As pointed out by [Ellickson and Misra \(2008\)](#), price setting is one of the most important yet tough challenges in the retail industry. For Jumbo Supermarket, it needs to have all competitor products linked to their Jumbo counterparts and monitor their prices regularly in order to uphold its lowest price guarantee. Also, it is necessary for all supermarkets to continuously compare the prices offered by its competitors against its own offerings to maximize profit while ensuring market share and be prepared for possible responses by rivals. This is when product matching comes into play.

For Jumbo Supermarket, the absence of universal identifiers in competitor products poses a challenge to product matching. Every Jumbo product is associated with a unique European Article Number (EAN), which is a unique sequence of numbers and letters representing a product and its attributes ([Black, 2018](#)). Product matching could have been a simple lookup process if we also had had the EANs of competitor products, but they are unfortunately not publicly accessible. The matching process currently relies on manual labor aided by computer-generated recommendations based on product title similarity. The pricing team would therefore benefit greatly from a fully or at least partly automated solution to the product matching problem that would save them time and resources.

There has been considerable research in the field of product matching using textual and image features in the absence of universal identifiers (see

Section 4), and the most prominent approach to this problem is to divide it into two sub-problems, namely feature extraction and binary classification. A typical product matching model like the one by [Ristoski, Petrovski, Mika, and Paulheim \(2018\)](#) is designed to determine whether two items are a match by extracting text and image features and feeding a list of cosine distances calculated from these features into a binary classification model.

However, there are some challenges to supermarket product matching that have not yet been fully addressed by previous research. Firstly, most of the previous research efforts focus on online retailers rather than the supermarket setting. Product matching for supermarkets shares many similarities with that in general but the unique concept of private label, which will be discussed in Section 4, sets it apart. Secondly, most current research efforts experiment on data with only a few selected product categories while the task of product matching can be more challenging for a supermarket with a much greater variety of products. In addition, there is a lack of large public datasets that contain not only multi-modal features but also true labels as pointed out by [Wilke and Rahm \(2021\)](#). The datasets supplied by Jumbo Supermarket are valuable for research in this aspect since they fulfill both criteria and cover a wide range of product categories. Research in this direction could therefore be a nice addition to the current endeavors.

More importantly, there is also room for improvement that can be observed upon reviewing existing research efforts. Firstly, recent research opens the door to applying newly developed embedding models to extract features for product matching purposes. Secondly, to the best of our knowledge, there has not yet been research dealing with product matching using a multi-label classification approach, which might give better matching results than the pairwise approach since the contextual information of other prospective candidates may be useful. Thirdly, the Jumbo datasets contain other features such as product size and brand name that may further improve product matching performance.

The main research question this thesis aims to answer is:

In what ways can supermarket product matching, as a supervised machine learning problem combining metric learning and classification, be improved?

To answer the main research question, three sub-questions will be explored:

- RQ1 *To what extent would metric learning for supermarket product matching benefit from transfer learning and multi-modal modeling?*
- RQ2 *How would multi-label classification compare to binary classification in the context of supermarket product matching?*

RQ3 *Apart from product title and image, what features are also useful for the matching of supermarket products?*

This research paper finds that transfer learning with triplet loss and multi-modal modeling can improve feature extraction results by creating more discriminative embeddings for similarity calculation, and multi-label classification model outperforms binary classification model when only cosine distances are provided. It is also observed that the performance of multi-label classification model can be further improved by adding size-related features.

The structure of the rest of this research paper is as follows. Section 4 describes the related work on product matching, Section 5 explains the feature extraction and classification models used in this research, Section 6 illustrates the experimental setup, Section 7 lists the results of the experiments, Section 8 discusses the implications of the results and finally Section 9 concludes this research paper.

4 RELATED WORK

Supermarket product matching is faced with many challenges. One of them is data limitation as there are often only a few instances for each product across all supermarkets (Wei, Tran, Xu, Kang, & Springer, 2020). There is also the requirement for flexibility since an ideal product matching algorithm should be resilient to the large number of new products released every day, and it should require as little retraining as possible whenever new products are introduced (Shah, Kopru, & Ruvini, 2018). Compared to ordinary product matching, supermarket product matching for pricing comparison also has an extra layer of complexity due to the concept of private label. Private label products are those manufactured and sold exclusively by a supermarket. They are not offered by other supermarkets but still need to be matched with competitor items that are intrinsically similar for the sake of price comparison. In other words, matching private label products is more than "entity resolution", which is to "identify representations referring to the same real-world entity" (Köpcke, Thor, & Rahm, 2010).

As discussed in Section 3, the mainstream approach to product matching consists of two main steps, namely feature extraction and binary classification. The related work in these two aspects is summarized as follows.

4.1 Feature Extraction

Many recent research efforts have investigated ways to improve the quality of text and image embeddings for specific product categories. The most common approach is using triplet loss, which is a loss function for similarity learning as explained in Section 5.3. For example, [Forssell and Janér \(2020\)](#) attempted to train a convolutional neural network with triplet loss for matching “accessories”, “clothes” and “shoes”. Some researchers opted for fine-tuning pre-trained models with triplet loss, since training a model from scratch requires a huge volume of labelled training data and extensive expertise in order for it to converge properly and generalize well ([Tajbakhsh et al., 2016](#)). For instance, [Tracz et al. \(2020\)](#) examined whether fine-tuning BERT model with triplet loss would boost its performance on matching products from the “culture”, “electronics” and “beauty” categories. Going one step further, [Ko \(2021\)](#) investigated whether fashion product matching would benefit from multi-modal modeling by training multi-modal embedding models with triplet loss on image and sentence embedding vectors extracted by pre-trained Xception and Sentence-BERT models respectively.

While triplet loss and multi-modal modeling have been proven to be useful for data with limited categories, it is still unclear whether they would perform as competently when faced with the great diversity of supermarket products. Also, there are some newly established pre-trained embedding models that can potentially boost the performance of product matching, such as the Universal Sentence Encoder (see Section 5.1). This research has therefore used some of these new models and applied triplet loss as well as multi-modal modeling on the Jumbo datasets with 15 main product categories to check their applicability to supermarket product matching.

4.2 Classification of Correct Matches

After transforming texts and images to numerical representations, one would also need to develop a classification model to give match results. A common approach to match classification is to compare the feature embeddings of two items and feed their similarity scores or distances to a binary classification model that determines whether they should be a match. For example, [Ristoski et al. \(2018\)](#) trained binary classification models that could determine whether two items are a match after receiving a “similarity feature vector”, which is a list of similarity scores based on text and image features extracted by conditional random field, paragraph2vec and convolutional neural networks. They experimented with four classification

algorithms on three selected product categories and concluded that random forest gave the best performance on classifying product pairs that the annotators manually selected. [Wilke and Rahm \(2021\)](#) adopted a similar binary classification approach on a dataset of shoe products but it relied solely on deep learning models to extract text and image embeddings. They compared the embeddings of two items to get vectors of element-wise absolute distances that were fed to a binary classification neural network model to determine whether they were a match.

However, the possibility of multi-label classification has not been explored in previous research. Current research efforts mostly dealt with multi-directional comparison, which is to identify identical products from various data sources by comparing them against each other. Meanwhile, product matching for Jumbo Supermarket is a one-way comparison, which requires the algorithm to compare competitor products against all available Jumbo products and identify matches if there is any. For one-way matching, a multi-label classification model with a fixed number of classes can be developed by selecting a fixed number of closest Jumbo products for each competitor product, calculating cosine distances for each pair, and feeding all distances into a multi-label classifier to distinguish which of them should be a match. This research therefore attempts both binary and multi-label classification to see if the extra contextual information about other prospective candidates can make the matching results more accurate.

In addition to cosine distances, some previous research also included product attributes in hope of improving the matching results. For instance, [Li, Dou, Zhu, Zuo, and Wen \(2020\)](#) leveraged product title and four other attributes, namely brand name, net weight, place of origin and function, to train binary classification models for matching products on e-commerce platforms. Apart from product title and image, the datasets used in this research contain other product attributes that are potentially useful for product matching, including volume, unit of measure and whether an item is branded or private label product. For example, branded products tend to have the same images across different supermarkets but private label products often have different packaging, and the classification model may thus assign less weight to the image part when dealing with private label products. It is also worthwhile to look at these attributes and check if including them in the final classification model would improve the results.

5 METHODS

In order to examine ways to improve supermarket product matching results, various machine learning and deep learning models have been explored in this research regarding feature extraction and classification.

This section is going to describe these models and explain the motivation behind choosing them.

5.1 *Universal Sentence Encoder*

The multi-lingual Universal Sentence Encoder (USE) is chosen for extracting sentence embeddings in this research. Developed by [Yang et al. \(2019\)](#), it is a sentence embedding model designed specially for multi-lingual transfer learning that has been proved to be successful in multiple applications. The encoder is based on the encoder part of the Transformer architecture, which is currently the “state-of-the-art architecture for natural language processing” ([Wolf et al., 2019](#)). According to [Vaswani et al. \(2017\)](#) who first proposed the architecture, the key concept of Transformer is self-attention. It allows word embeddings to be generated based on not only their meanings, but also their ordering and relationship with other words in the sentence. Compared to recurrent neural networks such as Long Short Term Memory network that are also commonly used for natural language processing, Transformer can also be faster and better at capturing very long-range dependencies due to its lack of recurrence. For the Universal Sentence Encoder, these context-aware word embeddings are then averaged to generate sentence embeddings as output.

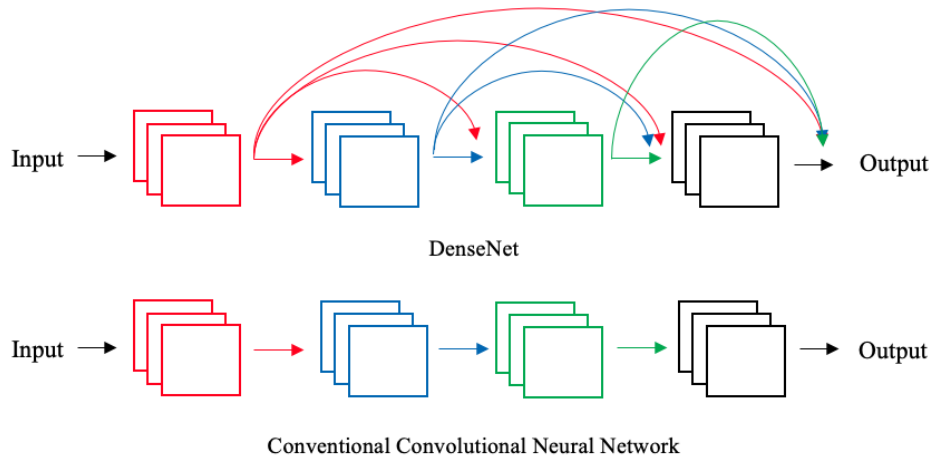
According to [Yang et al. \(2019\)](#), the multi-lingual USE available on TensorFlow Hub is pre-trained on texts in 16 languages, including Dutch and English that are the two most common languages of product titles that appear in the datasets. Similar to the Multi-task Dual-Encoder Model proposed by [Chidambaram et al. \(2018\)](#), the multi-lingual USE is trained by using a shared encoder for question-answering, natural language inference and translation ranking and optimizing the model one task at a time in an iterative manner. While allowing flexibility in input language, the multi-lingual encoder achieves competitive results compared to mono-lingual encoders in various tasks, including identifying semantically similar sentences. The simplicity, flexibility and high performance of the multi-lingual USE have motivated the choice of using it for this research.

5.2 *Densely Connected Convolutional Networks (DenseNet)*

Pre-trained DenseNet-201 model is chosen for extracting image embeddings in this research. DenseNet is a convolutional network architecture first proposed by [Huang, Liu, Van Der Maaten, and Weinberger \(2017\)](#) to alleviate problems such as vanishing gradient in a deep convolutional network. A typical DenseNet model consists of multiple dense blocks and each layer in a dense block takes all previous feature maps in that

dense block as input, instead of only the immediately preceding feature map like a conventional convolutional neural network layer. Such dense connectivity facilitates feature reuse and maximizes information flow, thus allowing greater depth in the resulting neural network (Hess, 2018). Due to its competitive performance on image-related deep learning applications, DenseNet is one of the models that are often adopted by researchers as baseline model or the base model for transfer learning in image-related tasks (Forssell & Janér, 2020), which is the reason that it is chosen for this research.

Figure 1: Comparison of forward propagation between a dense block in DenseNet and a conventional convolutional neural network



DenseNet-201, the model chosen for this research, is a deep learning model of 201 layers based on the DenseNet architecture. It is available on TensorFlow with pre-trained weights learned from classifying 1.2 million images from the ImageNet dataset with 1,000 possible classes (Gupta & Murzova, 2017). The fully connected output layers of the pre-trained model can be removed and global average pooling can be applied to the output of the last convolutional block to generate image embeddings for various transfer learning applications (Abadi et al., 2021).

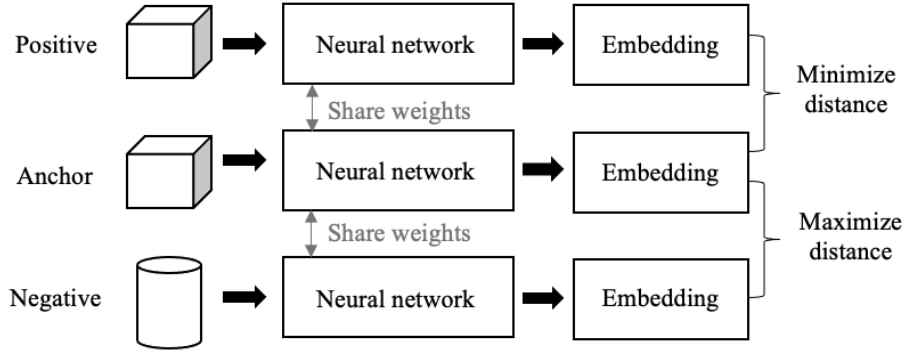
5.3 Triplet Loss

Triplet loss is a loss function for similarity learning by minimizing the distances between samples from the same classes and maximizing those between samples from different classes as suggested by Schroff, Kalenichenko, and Philbin (2015). The loss function does not require a fixed number of classes as opposed to softmax loss that is usually used for multi-class

classification. Such flexibility in training data suits the ever-changing and extremely diverse product offerings provided by retailers. It has therefore been widely used for distance metric learning by previous product matching research efforts, including the ones by [Tracz et al. \(2020\)](#) and [Forssell and Janér \(2020\)](#).

Triplet loss is typically used in a Siamese neural network consisting of three identical neural network models that share weights. The network takes as input an anchor, a positive sample with the same identity as the anchor, and a negative sample with a different identity from the anchor.

Figure 2: Visualization of a Siamese neural network with triplet loss

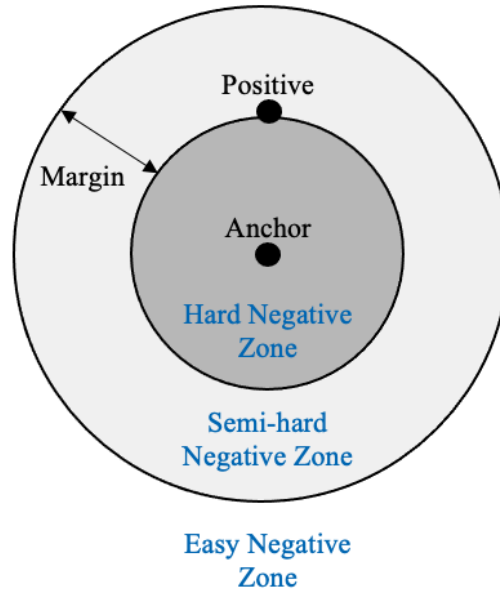


After the three samples pass through the neural network models, the loss will be calculated based on the L2 normalized Euclidean distance between the anchor and the positive sample as well as that between the anchor and the negative sample using the following formula:

$$L(A, P, N) = \max(0, \|f(A) - f(P)\|_2^2 - \|f(A) - f(N)\|_2^2 + m) \quad (1)$$

where A is the anchor, P is the positive sample, N is the negative sample, $f(x)$ is the neural network model with shared weights, and m is the margin. Margin is a hyperparameter that determines whether a negative sample is sufficiently far from the anchor compared to the positive sample that no weight adjustment is needed. As summarized by [Wang, Liang, and Wang \(2021\)](#), there are three possible kinds of negative samples for a fixed pair of anchor and positive samples. The first possibility is easy negative, which is further away from the anchor than the positive and the difference in distances is larger than the margin. The second one is semi-hard negative, which is also further away from the anchor than the positive but the difference in distances is smaller than the margin. The last one is hard negative, which is closer to the anchor than the positive.

Figure 3: Visualization of the three types of negatives



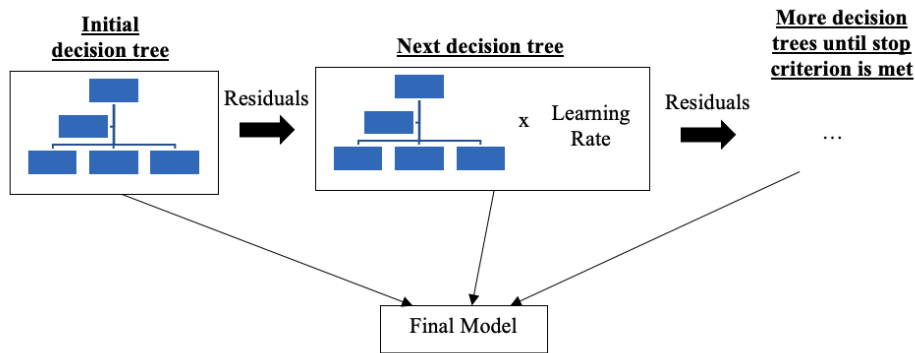
Out of the three types of negatives, semi-hard negatives are used for fine-tuning pre-training models in this research. According to the FaceNet research by [Schroff et al. \(2015\)](#), easy negatives do not help the training and would slow down convergence. Selecting only hard negatives could lead to bad local minima and even a collapsed model while using semi-hard negatives gave the best results. Subsequent research by [Harwood, Kumar BG, Carneiro, Reid, and Drummond \(2017\)](#) echoed the finding and suggested that even though the hard negative only mining approach sometimes led to the best results, it did often cause model collapse. In contrast, using only semi-hard negatives led to a steadier learning curve and gave competitive results in distance metric learning tasks. In view of its stable performance and the already existing TensorFlow implementation that makes it easy to use ([Moindrot, 2018](#)), online semi-hard triplet mining is used together triplet loss in this research for fine-tuning pre-trained text and image embedding models as well as training a multi-modal embedding model.

5.4 *LightGBM*

Light Gradient Boosting Machine is the classification model chosen for this research. It is based on gradient boosting decision tree (GBDT) proposed

by [Friedman \(2001\)](#). As a machine learning technique of training weak decision trees in an iterative manner, GBDT is built by fitting one decision tree on the residuals of the trees trained before it with the gradient descent optimization algorithm at a given learning rate, and eventually ensembling all trained decision trees to give predictions. The boosting technique and the algorithms derived from it have been successful and achieved high performance in various machine learning scenarios as suggested by [Hang, Huang, Cai, Yang, and Lin \(2021\)](#).

Figure 4: A graphic illustration of the training process of the GBDT model



Developed by [Ke et al. \(2017\)](#), LightGBM improves on GBDT by introducing two concepts. The first one is Gradient-based One-Side Sampling, which is down sampling by retaining all instances with high gradients and randomly selecting a subset of those with low gradients. The second one is Exclusive Feature Bundling, which is feature reduction by bundling features that do not or almost do not have non-zero values simultaneously. They aim to reduce the computational complexity of the GBDT algorithm while giving approximately the same accuracy. In addition, its Python implementation comes with built-in support for missing value handle by representing them with NaN and automatic adjustment of class weight to tackle the problems of missing data and class imbalance, both of which can be found in the datasets used in this research. Due to its simplicity, speed and high performance, LightGBM is chosen as the machine learning algorithm for the classification models explored in this research.

6 EXPERIMENTAL SETUP

This section will explain the datasets used in this research, how they are pre-processed as well as how the experiments are implemented and evaluated.

6.1 Raw Datasets

There are three datasets used in this project and they are for internal use within Jumbo Supermarket only, which means they have never been used in research before. They are all stored in Delta table format on Azure cloud server and can be easily converted to CSV or other common file types.

The first dataset contains 18,826 active Jumbo products that were available on the shelf as of 5 August 2021. As shown in Table 4 in Appendix A, there are various features included in the dataset, the most important of which are product title and product image path that are used to train embedding models. There are also other features such as brand name, unit of measure and volume that can be added to the final classification model to boost its performance. As far as missing data is concerned, most columns do not have missing values, except the image path columns with 244 entries having no images at all and the "topLevelCategoryId" column with 218 missing values.

The second dataset contains 52,482 entries of competitor products. A competitor product can have multiple entries if it is matched with more than one Jumbo counterpart, and in that case those entries will be identical except the "jumbo_id" column. As shown in Table 5 in Appendix A, the dataset has 13 features, including the two key features, product title and product image path, and the Jumbo counterpart which serves as the true label for matching. It also contains features such as brand name, unit of measure and volume that also appear in the Jumbo product dataset and can be used as extra features in the final classification model. Similar to the Jumbo dataset, the majority of the columns in the competitor product dataset do not have any missing values, except the "comp_catg_desc" column with 6,016 missing values, the three size-related features with around 5,500 missing values each, the image path column with 4,283 missing values and the "comp_article_status" column with 5,486 missing values.

The third dataset contains the names of 182 private labels from all Jumbo's competitors in the Netherlands and their equivalent Jumbo private labels. The brand name columns of the first two datasets can be compared against this table to create a binary private label indicator, which could be a useful feature as explained in Section 4.2.

6.2 Data Pre-processing

The three raw datasets mentioned above have been pre-processed into two final datasets by selecting only useful features and cleaning them before using.

The first dataset contains Jumbo active products and its overview can be founded in Table 1. The "jumbo_pl" feature is a binary private label product indicator created by comparing the brand name column against the dataset of private label brands. Also, it is observed that the unit of measure column only has five possible values, which are "GR" (short for gram), "KG" (short for kilogram), "ML" (short for milliliter), "WB" (short for wasbeurten in Dutch or number of washes in English), and "ST" (short for stuk in Dutch or piece in English). To standardize the units even further, all entries with "KG" as unit of measure are converted to "GR". In addition, the two columns of image paths are merged into one since photos in primary view and other views are both useful for matching. Finally, the two category-related columns are dropped since every supermarket has a different set of product categories and it is decided not to formulate a universal category scheme that may not turn out to be useful due to time constraint for this research.

Table 1: Features in the pre-processed Jumbo product dataset

Feature	Type	Description
jumbo_id	string	Unique identifier of Jumbo product
text	string	Product title
image	array of strings	Path(s) to product image(s)
jumbo_brand_name	string	Name of brand
jumbo_pl	boolean	Whether the product is a private label product
jumbo_volume	float	Number of units of measure contained in the product
jumbo_uom	string	Unit of measure

The second dataset contains competitor product and its overview can be founded in Table 2. First of all, to get distinct competitor products that can be matched with the first dataset, entries with Jumbo targets that are not present in the Jumbo product dataset are removed, and the remaining entries are grouped by the column of Jumbo target for de-duplication. The resulting dataset consists of 46,565 unique competitor products each with one or more matches in the "target_id" column. Secondly, some values in the volume column are not numbers but expressions like "20 x 4" to indicate the volume per portion. The Python library `simpleeval` is used to convert these expressions into numbers. Thirdly, to align with the Jumbo product dataset, the units of measure of competitor products, which take 33 distinct values in total, are converted into one of the four possible values – "GR" for gram and its multiples, "ML" for milliliter and its multiples, "WB" for number of washes, or "ST" for all other units – with the help of approximate string matching powered by the library `difflib` in base Python. The volume column is also adjusted accordingly. Finally, the column of private label indicator is added and the category column is dropped for

the same reason as the Jumbo dataset. Several columns of duplicated identifiers are also dropped.

Table 2: Features in the pre-processed competitor product dataset

Feature	Type	Description
comp_article_ext_id	string	Identifier of competitor product
comp_banner_id	integer	Identifier of competitor
comp_article_id	integer	Internal reference to competitor product
comp_banner_name	string	Name of competitor
comp_article_desc	string	Product title
img_dir	string	Path to image of the product
target_id	array of strings	Unique identifier(s) of Jumbo counterpart(s) with which the product is matched
comp_brand_name	string	Name of brand
comp_pl	boolean	Whether the product is a private label product
comp_volume	float	Number of units of measure contained in the product
comp_uom	string	Unit of measure

Textual features and images in these two final datasets are also pre-processed to make sure they are in the right formats expected by the USE and DenseNet models. As for textual features, some of them contain garbled text characters, mostly French and German special characters for brand names. The Python library `ftfy` (Speer, 2019) is used to make sure text characters are decoded correctly as UTF-8. As for product images, all images are converted to RGB format and resized to 224x224 and their image pixels are scaled to the range of zero to one to meet the input requirements of the DenseNet model.

6.3 Experimental Procedure

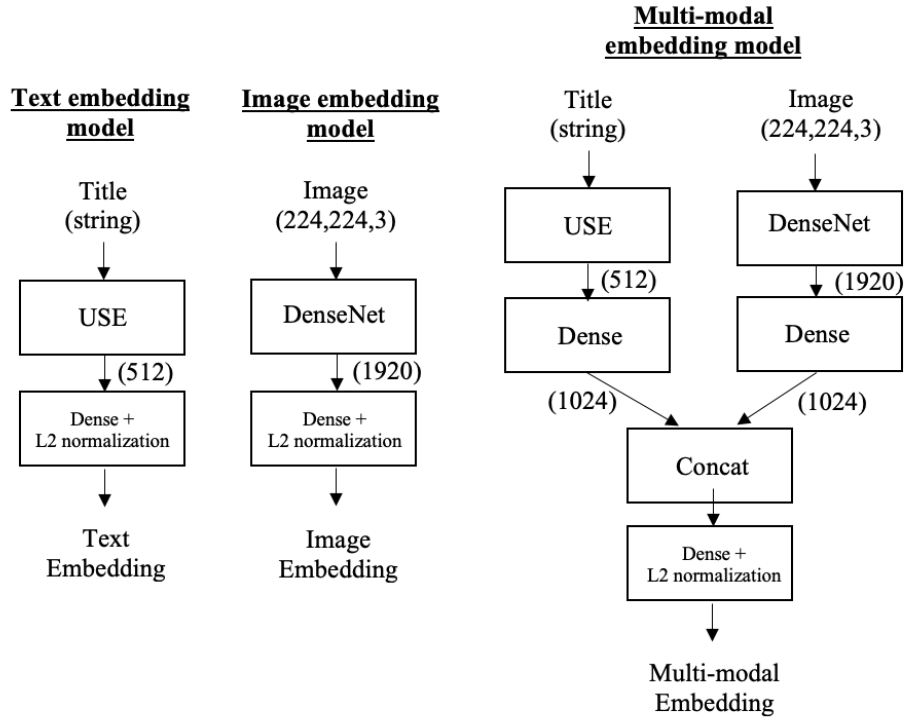
All experiments in this research have been conducted on Azure Databricks¹ with a 32-core CPU cluster with 224GB memory in total and a dual-core GPU cluster with 224GB memory in total. For evaluation, the competitor product dataset is divided into training, validation, and test sets in a 70:15:15 ratio. In order to prevent test-train overlap that would lead to overly optimistic results and to make sure the trained models also work on completely unseen products, the dataset is grouped by the “target_id” column before train-validation-test split to ensure the diversity of products between the three sets. The three sets of data consist of 32,485, 7,012 and 7,068 competitor products respectively. The code for our experiments is written in Python 3 (Van Rossum & Drake, 2009).

¹ Azure Databricks is a data science platform by Microsoft that offers users with a Jupyter notebook-like coding environment that fully supports Apache Spark.

6.3.1 Distance Metric Learning

The first sub-question concerns whether triplet loss and multi-modal modeling would improve the quality of embeddings generated by deep learning models. To answer this sub-question, three models are trained and compared to pre-trained models as shown in Figure 5.

Figure 5: Conceptual overview of the distance metric learning models trained in this research



To speed up training in the limited time available for this research, we add additional fully connected layers to pre-trained models to achieve pseudo model tuning as suggested by Miller, Howard, Adams, Schwan, and Slater (2020). The text model is the pre-trained multi-lingual Universal Sentence Encoder (Yang et al., 2019) available on TensorFlow Hub² with a dense layer added. The image model is the pre-trained DenseNet-201 model available on TensorFlow³ with global average pooling applied to make the output one-dimensional and the original fully connected layers of 1,000 classes replaced by a new dense layer. Based on the model structure proposed by Miller et al. (2020), the multi-modal model first runs text and

² <https://tfhub.dev/google/universal-sentence-encoder-multilingual-large/3>

³ https://www.tensorflow.org/api_docs/python/tf/keras/applications/densenet/DenseNet201

image through their respective pre-trained models and additional fully connected layers, and then concatenates the two embeddings into one and passes it to another fully connected layer. A L2 normalization layer is also added to all models as prescribed by [Schroff et al. \(2015\)](#).

Top-1 accuracy is adopted as the main evaluation metric in line with previous research efforts on metric learning for product matching including the ones by [Tracz et al. \(2020\)](#) and [Forssell and Janér \(2020\)](#). When evaluating a model, an embedding will be generated for each competitor product, and it will be compared against the embeddings of all Jumbo products to find the closest Jumbo match in terms of cosine distance using the nearest neighbour search algorithm⁴ in the scikit-learn package ([Pedregosa et al., 2011](#)). The predicted matches will be compared against the true labels and the top-1 accuracy here is defined as the proportion of correctly matched results.

For training the three models, Adam ([Kingma & Ba, 2014](#)) is used as the optimizer and triplet loss with semi-hard triplets mining available on TensorFlow Addons⁵ is used as loss function due to reasons explained in Section 5.3. The loss function expects the input to be embeddings and their corresponding labels in integer format. To achieve this format, a competitor data subset has its “target_id” column exploded, which is to return a new row for each element in the list, and is then merged with Jumbo products that appear in the competitor data subset using the union function. The Jumbo ID column is then encoded into label indices using StringIndexer⁶ in PySpark ([Zaharia et al., 2016](#)). Since the pre-trained models are not fine-tuned directly and remain unmodified during the training process, we feed the models with pre-trained embeddings directly to further speed up training. A graphic illustration of the data engineering can be found in Figure 6.

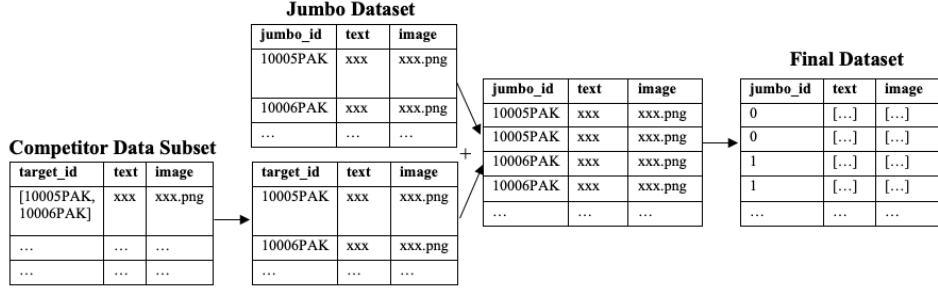
In addition, grid search is used for hyperparameter tuning and early stopping is applied to prevent overfitting. A model will first be trained for 10 epochs on all combinations of available hyperparameters (learning rate, margin, batch size and size of the fully connected output layer). The model is then trained on the set of hyperparameters with the best evaluation performance for 100 epochs and the optimal number of epochs is determined by looking at the trend of accuracy and loss for both training and validation data. A final model is trained on the best set of hyperparameters for the optimal number of epochs and tested on the test data.

⁴ <https://scikit-learn.org/stable/modules/generated/sklearn.neighbors.NearestNeighbors.html#sklearn.neighbors.NearestNeighbors>

⁵ https://www.tensorflow.org/addons/api_docs/python/tfa/losses/TripletSemiHardLoss

⁶ <https://spark.apache.org/docs/latest/api/python/reference/api/pyspark.ml.feature.StringIndexer.html>

Figure 6: Overview of data pre-processing for triplet loss



The three trained models are compared to their corresponding baseline models in terms of test accuracy. The baseline models for text and image are the pre-trained multi-lingual Universal Sentence Encoder and DenseNet-201 model respectively, and embeddings from the multi-modal model are compared against the concatenated embeddings from the two pre-trained models.

6.3.2 Classification Models

The second and third research questions concern the classification part of the product matching problem.

To answer the question of whether binary or multi-label classification works better, a binary classifier and a multi-label classifier, both with competitor product as input and Jumbo product as output, are trained using the LightGBM machine learning algorithm (Ke et al., 2017). In order to get a fixed number of classes for multi-label classification to work, the nearest neighbour search algorithm in the scikit-learn package (Pedregosa et al., 2011) is used to find the ten closest Jumbo products for each competitor product in terms of cosine distance with the concatenation of the three available embeddings. If a competitor product does not have any image, only text embeddings will be used for nearest neighbour search. The candidates are also sorted in alphabetical order to prevent a very imbalanced label distribution with the first candidate always being a correct match.

A multi-label model is chosen because a competitor product can be matched with one or more Jumbo products according to the available data. Since LightGBM does not have native support for multi-label output, the MultiOutputClassifier function⁷ in scikit-learn (Pedregosa et al., 2011) is

⁷ <https://scikit-learn.org/stable/modules/generated/sklearn.multioutput.MultiOutputClassifier.html>

used to extend it to multi-label classification by fitting one binary classifier per target.

Exact match ratio is adopted as the main evaluation metric since incorrectly matched products are not desirable for the given business problem, be it false positive or false negative. In the context of this research, both classification models will determine whether each of the top-10 candidates for every competitor product is a match or not and the classification results for all ten candidates are compared with the ground truth. A prediction is only considered correct when all ten results exactly match the target labels, and partial matches are considered incorrect. However, precision and recall are also taken into consideration in this research since they can also offer useful information regarding where the models usually go wrong. Since the ten classes representing candidates are arbitrarily defined and we are more interested in understanding the classifier’s performance on individual pairs of competitor and Jumbo products, micro-averaged instead of macro-averaged measures are adopted.

For training classification models, the multi-label classifier is fed with 30 cosine distances, three for each Jumbo candidate with respect to its text, image and multi-modal embeddings compared against those of the input competitor product, while the binary classifier only takes three cosine distances as input, which are the cosine distances between the three embeddings of a Jumbo product and those of a competitor product. Missing values are retained and fed to the model since LightGBM has native support for null values. To compare the results easier, the binary classifier is only tested on the top-10 candidates instead of all available Jumbo products. In view of class imbalance in the data, the “is_unbalance” parameter of the models is set to True so the model can automatically adjust class weight to prevent the algorithm from ignoring the minority class. To create reproducible results, the “deterministic” and “force_col_wise” parameters of the models are also set to True. Data sub-sampling is also enabled to speed up training and prevent overfitting.

In addition, grid search and five-fold cross-validation used for hyperparameter tuning with the help of the GridSearchCV function⁸ in scikit-learn. Since the training time of LightGBM is much shorter than deep learning models, cross validation is adopted instead of hold-out validation and the original training and validation sets are combined into one training set. A model will be trained on all combinations of some selected hyperparameters (number of iterations, learning rate, maximum number of bins, maximum number of tree leaves and maximum tree depth). A final model

⁸ https://scikit-learn.org/stable/modules/generated/sklearn.model_selection.GridSearchCV.html

is trained on the set of hyperparameters that achieve the best evaluation exact match ratio and tested on the test data.

After deciding whether binary or multi-label classification works better, the best performing model will be re-trained to take into account three kinds of extra features. The first feature is the ratio of the volume of the candidate to that of the competitor product, the second one is a binary feature indicating whether the unit of measure of the Jumbo candidate and that of the competitor product are the same, and the third one is a binary feature indicating whether a product is a private label product or not.

To assess whether these features would help further improve the chosen metrics, three other classification models are trained and compared to the original multi-label classifier. The first model uses the two size-related features only, which will be a binary classifier with 5 input features or a multi-label classifier with 50 features. The second one includes binary features indicating whether the competitor product and the Jumbo candidates are private label products respectively, which will be a binary classifier with 5 input features or a multi-label classifier with 41 features⁹. The third model includes all three kinds of extra features mentioned above, which will be a binary classifier with 7 input features or a multi-label classifier with 61 features. The evaluation methods and hyperparameter tuning process are the same as mentioned above.

7 RESULTS

In this section, the performance of embedding models and classifiers described in Section 6.3 will be presented.

7.1 Distance Metric Learning

Table 3 shows the performance of the three embedding models trained with triplet loss on the test data compared to that of their respective baselines. All three models outperform their baseline counterparts after training, both on branded products and on private label products, in accordance with previous research on product matching with fewer product categories. The results show that triplet loss is a viable approach to fine-tune embedding models even in a supermarket scenario with diverse product types.

In terms of top-1 accuracy, the multi-modal model achieves the best overall results (74.74%) and also works the best on branded products (83.91%). Interestingly, the text embedding model performs the best on

⁹ There are 10 Jumbo candidates but only one competitor product, so there are 11 extra private label indicators on top of the 30 features we use to train a multi-label classifier.

Table 3: Top-1 test accuracy of models trained with triplet loss compared to their baseline models

	Model	Overall Top-1 Test Accuracy	Top-1 Test Accuracy for Branded Products	Top-1 Test Accuracy for Private Label Products
Text	Pre-trained USE	54.39%	62.34%	33.66%
	Trained Model	69.08%	73.84%	56.57%
Image	Pre-trained DenseNet-201	63.80%	77.11%	28.38%
	Trained Model	67.93%	81.63%	31.51%
Multi-modal	Concatenated Pre-trained Embeddings	63.98%	77.33%	28.50%
	Trained Model	74.74%	83.91%	50.34%

private label products with a top-1 accuracy of 56.57%, but it is immediately followed by the multi-modal model with a top-1 accuracy of 50.34%. Multi-modal modelling is therefore proven to be an effective approach to summarize textual and visual information and create even more discriminative embeddings in most cases compared to mono-modal strategies.

Compared to the text embedding model, the image embedding model works better on branded products but less competently on private label products. Upon manual inspection of product images, it appears that the image embedding model’s poor performance on private label products can be attributed to the fact that their packaging designs tend to be very different across supermarkets while branded products usually have the same or very similar images across different supermarkets.

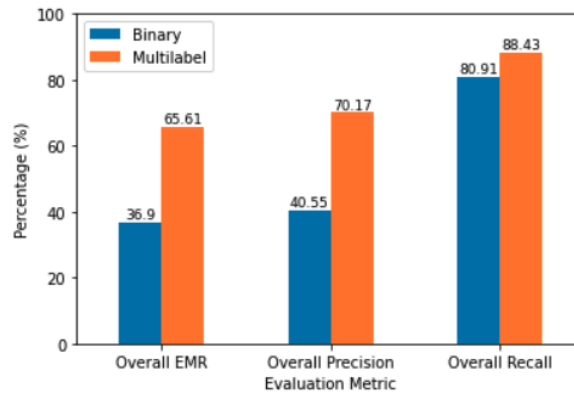
7.2 Classification Models

Before examining the proposed multi-label classification approach, we must first confirm that true matches exist among the top-10 candidates most of the time, otherwise the resulting model may not be able to learn how to find true matches because they are not there. Fortunately, the top-10 candidates of a product include all its true match(es) 97.90% and 96.86% of the time in the training set and test set respectively. The proportion of entries with their true matches partially or fully not included in the top-10 candidates is small and it is not possible for a model to get a high exact match ratio by always outputting no matches.

As to whether binary or multi-label classification works better for supermarket product matching, Figure 7 shows that the multi-label classifier outperforms its binary counterpart in all of the three chosen evaluation metrics. It gives an exact match ratio of 65.61% with only three types of distances while the binary classifier only achieves an exact match ratio of 36.90%. The low exact match ratio of the binary classifier does not mean it is useless. The performance of the two classifiers is close in terms of micro-averaged recall, but a huge difference is observed when it comes to micro-averaged precision, which hints that the binary classifier tends

to detect much more matches than it should compared to its multi-label counterpart.

Figure 7: Comparison of binary and multi-label classifiers in terms of the chosen evaluation metrics



After finding out that the multi-label classifier works better than the binary classifier on the test data, the former has been re-trained with extra features to see if they can improve its performance. As shown in Figure 8, adding size features to the classifier improves its exact match ratio by 4.03% and micro-averaged precision by 4.95%. In comparison, private label indicators only slightly improve the exact match ratio of the original classifier by 0.21% and that of the classifier with size features by less than 1%. The results show that adding size features can clearly boost the performance of the classifier while private label indicators may not improve the classification results as effectively as expected. One possible explanation can be that product sizes are often not mentioned in product titles and may thus give the classifier extra information. On the other hand, brand names are almost always included in product titles and the embedding models should be able to learn that private label brands are semantically similar in the context of product matching even though they are literally different.

Figure 8: Comparison of the four different multi-label classifiers in terms of the chosen evaluation metrics

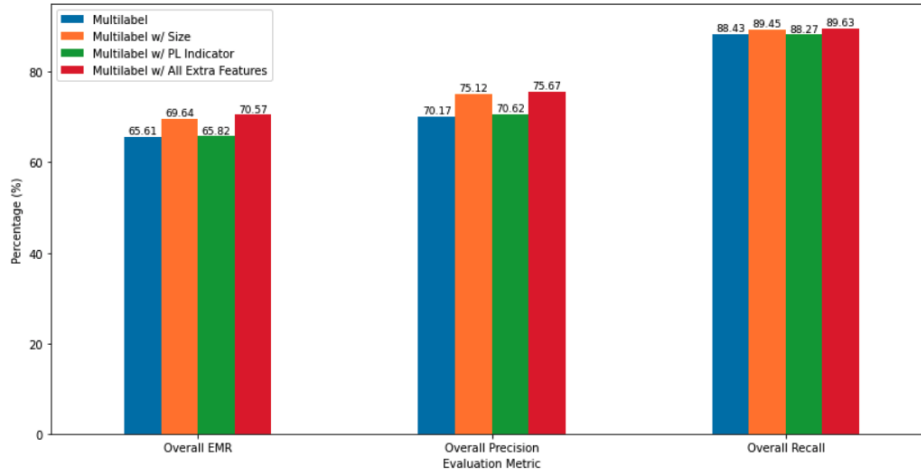
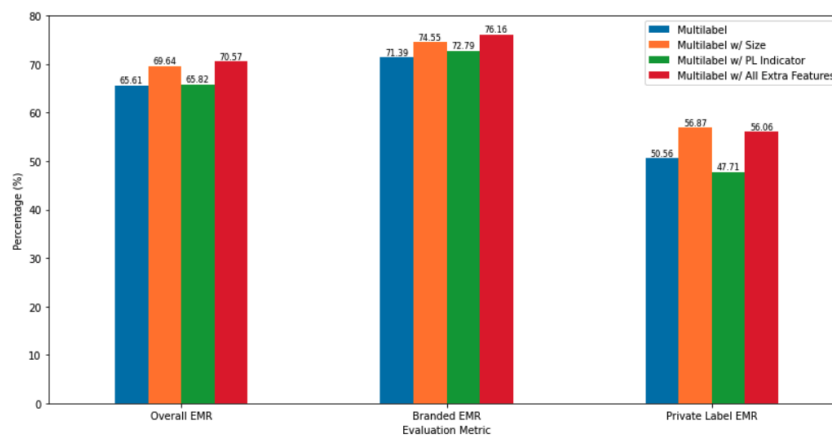


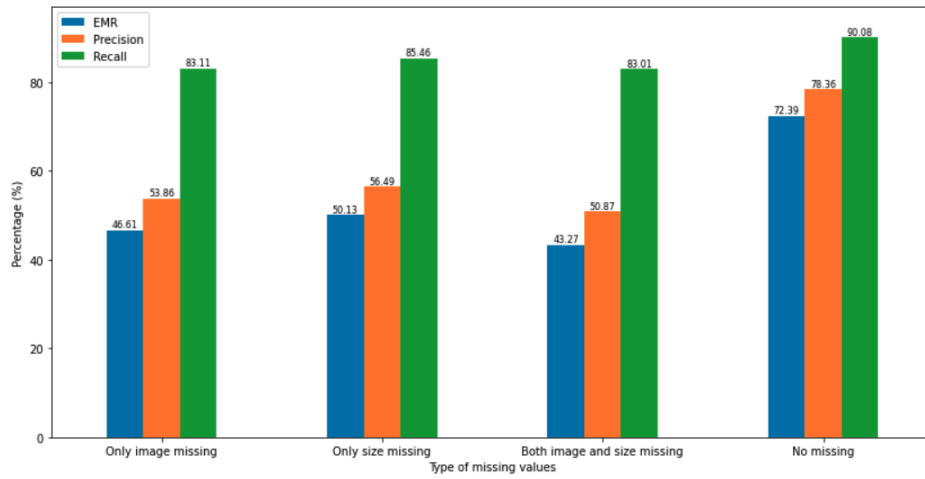
Figure 9 shows the performance of the four multi-label classifiers on branded and private label products respectively. Similar to the results in the embedding generation part, it can be observed that all classifiers perform better on branded products while private label products are more challenging to all of them. One plausible reason for the drop in performance can be the decrease in embedding quality when it comes to private label products as shown in Section 7.1 that makes cosine distances less meaningful for the classification model. Overall speaking, among the classifiers tested in this research, multi-label classifier with cosine distances and size features as input achieves a balance in maximizing the chosen evaluation metrics and keeping the feature space compact.

Figure 9: Comparison of the four different multi-label classifiers in terms of their exact match ratio on branded and private label products



Besides performance, the robustness of multi-label classifier to missing values is also being examined in this research. In the two final datasets used in this research, missing values are only observed in the product image and size-related columns. As shown in Figure 10, although the LightGBM has built-in support for missing values and can still perform matching with missing values, the performance of the multi-label classifier with cosine distances and size features as input drops significantly in terms of exact match ratio when one or more features are missing. It is therefore still important to have all required features for the model to achieve its best performance.

Figure 10: Comparison of the performance of multi-label classifier in case of missing values



The table containing the evaluation metrics of all classification models tested in this research can be founded in Table 6 in Appendix B.

8 DISCUSSION

The goal of this thesis paper is to examine ways of improving supermarket product matching using different techniques on the datasets containing product title, image and other characteristics of a product.

The first sub-question deals with embedding generation. The results show that with comprehensive datasets that cover a great variety of products, triplet loss with semi-hard triplet mining and multi-modal modeling are feasible and effective in improving embedding quality in terms of top-1 accuracy even in a supermarket setting. Not only does fine-tuning with triplet loss improve the discriminative power of embeddings and make distances between them more meaningful, it can also be used to reduce

the embedding size and thus the computational cost for nearest neighbor search. For example, a pre-trained DenseNet-201 embedding consists of 1,920 float numbers while the size of a fine-tuned image embedding is only 512, which is only 26.67% of the original. The multi-modal model has also demonstrated its ability in aggregating textual and visual information to generate embeddings with higher quality than using only texts or images. Their outstanding performance echoes the findings of previous research.

The high embedding quality is also manifested by the quality of top-10 candidates. As mentioned in Section 7.2, more than 96% of the top-10 candidate lists include all true matches. This means that even if the classification model does not work, supermarkets can still get a reliable list of candidates with the embeddings and greatly reduce the search scope for manual matching.

There is certainly room for further improvement in the distance metric learning part. This research has only explored pseudo model tuning due to time constraint. Despite its satisfactory performance, it might be possible to further enhance embedding quality by unfreezing some layers of the pre-trained models and fine-tuning them at the expense of longer training time.

Also, some regularization methods such as transfer learning, early stopping and L2 norm penalty are adopted to increase the generalizability of the embedding models, but other methods such as data augmentation have not been explored in this research. As explained by Shorten and Khoshgoftaar (2019), data augmentation is to create synthetic samples by transforming the original instances, by means such as cropping and scaling for images and random noise insertion for texts, while preserving their original labels. It is a way to allow a deep learning model to deal with variability and improve its generalization performance. Although no significant overfitting is observed in the experimental results, it may be interesting for future research efforts to examine whether data augmentation would improve the generalization performance of the embedding models in the supermarket context where the majority of the products are branded products and there is not much variability in terms of product titles and images for them across supermarkets.

After finding ways to improve embedding quality, the remaining two sub-questions aim to improve the classification results. The second sub-question concerns whether the additional contextual information would help multi-label classifier outperform its binary counterpart with only cosine distances as input. The results are very positive, showing that the multi-label classifier performs much better than the binary classifier specifically in terms of exact match ratio and micro-averaged precision. The huge difference in precision suggests that the extra contextual information

helps the multi-label classifier give much fewer false positives than the binary one. It is clear that multi-label classification is indeed a viable and effective alternative to binary classification in the context of one-way supermarket product matching.

It is worth mentioning that multi-label classification is not the only way to include contextual information. For example, besides cosine distances, one can include the similarity ranking of a Jumbo candidate compared to other candidates in the classifier. One may also use recurrent neural network models such as long short-term memory network to capture the dependencies between candidates. However, there is no doubt that multi-label classification is an easy and fast solution to achieve that without the need to generate extra features.

It is also important to bear in mind that the one-way matching scenario explored in this research is different from previous research efforts. It may be more difficult to define the number of candidates in the case of multi-directional matching with the objective of identifying matching products from multiple data sources. When the number of candidates increases, the number of features would also increase accordingly and the high-dimensional data may constitute the curse of dimensionality, which may lead to slower computational speed and undermine the model's performance when there is not enough data. The multi-label classification approach may therefore not achieve such impressive results in multi-directional product matching that most previous research has explored.

The third sub-question aims to explore whether the two kinds of extra features regarding size and private label would further improve the performance of the multi-label classifier. The results show that size-related features can increase exact match ratio and micro-averaged precision by as much as 4% while private label indicators may not be very useful in enhancing the chosen evaluation metrics. Further analysis can be done to determine the reasons that private label indicators do not help improve the matching results that much, but adding only size features appears to be a reasonable trade-off between accuracy and feature size.

However, there can be more features that are useful in improving the results and they have not been explored due to their absence in the datasets used in this research. For example, product category may give the classifier extra information when properly cleaned, and product prices may also be able to tell the classifier to reject matches when the price difference is too huge. It is also possible to use these extra features for post-processing instead of including them directly in the classifier. These are all directions in which future research efforts can go in order to further improve the product matching results.

In general, it is clear that the classifiers perform less well when the input competitor product is a private label one or some features are missing. As a characteristic of supermarket products, the existence of private label products makes product matching more difficult since they are meant to be matched with intrinsically similar products instead of the same products provided by other supermarkets, and they have been proven to be a real challenge by the results of this research. The decrease in performance can be attributed to the decrease in embedding quality when it comes to private label products, as well as the scarcity of private label instances, which only account for approximately 20% of the datasets. It might be worthwhile for future research to treat private label products as an individual topic, and to investigate the reasons why they are difficult to match as well as ways to improve the matching results specifically on them.

Missing values also pose a great challenge to supermarket product matching and could lead to a 20-30% decrease in exact match ratio. There are some possible ways to potentially alleviate this problem, one of which is to extract product attributes from product titles or descriptions as presented by Ghani, Probst, Liu, Krema, and Fano (2006) to fill the missing values. Besides imputing missing values, users of the matching algorithm might consider manually matching only entries with missing values in the required columns, which account for only 12.9% of the given data, with the aid of top-10 candidates to guarantee confident results.

9 CONCLUSION

This paper aims to refine the traditional approach to product matching and adapt it to the one-way matching scenario for a supermarket. This primary motivation leads to the main research question of *"In what ways can supermarket product matching, as a supervised machine learning problem combining metric learning and classification, be improved?"* and the three sub-questions that drive the research in three possible ways to improve it.

For the first sub-question on the effectiveness of triplet loss and multi-modal modeling on supermarket data, transfer learning with triplet loss and multi-modal modeling have been proven to be applicable not only to a few selected product categories but also supermarket products that come with private label products and from as many as 15 product categories. Their usefulness is manifested by the improved discriminative power of embeddings and smaller embedding size.

For the second sub-question on binary and multi-label classification, the results show that multi-label classification provides more accurate results in terms of exact match ratio when compared to binary classification and is therefore preferred in the case of one-way matching.

For the third sub-question on extra features, size features such as volume ratio and unit match indicator do help further improve matching results while private label indicators are not that effective in comparison. It is concluded that a multi-label classifier with cosine distances and size features as input balances the need to maximize the chosen evaluation metrics and to reasonably limit the number of features.

Further research has to be conducted to investigate ways to further enhance embedding quality for private label products and include more product attributes, or exploring the feasibility of multi-label classification in the case of multi-directional matching. But in response to the main research question, a refined one-way supermarket product matching workflow is proposed as follows. After taking a competitor product as input, the first step would be to generate text, image and multi-modal embeddings with the three models fine-tuned with triplet loss. At the second stage, the top-10 closest candidates will be selected among the supermarket's own products in terms of cosine distance based on the three types of available embeddings. The multi-label classifier will then be provided with their cosine distances, volume ratio and unit match indicator in relation to the input competitor product and output the predicted matches.

REFERENCES

- Abadi, M., Agarwal, A., Barham, P., Brevdo, E., Chen, Z., Citro, C., ... Zheng, X. (2021). *tf.keras.applications.densenet.densenet201*. Retrieved 2021-11-19, from https://www.tensorflow.org/api_docs/python/tf/keras/applications/densenet/DenseNet201 (DenseNet-201 model available on tensorflow.org)
- Black, J. (2018). *European article numbering: Everything about ean codes*. Retrieved 2021-11-29, from <https://prisync.com/blog/ean-code/>
- Chidambaram, M., Yang, Y., Cer, D., Yuan, S., Sung, Y.-H., Strope, B., & Kurzweil, R. (2018). Learning cross-lingual sentence representations via a multi-task dual-encoder model. *arXiv preprint arXiv:1810.12836*.
- Ellickson, P. B., & Misra, S. (2008). Supermarket pricing strategies. *Marketing science*, 27(5), 811–828.
- Forssell, M., & Janér, G. (2020). *Product matching using image similarity*.
- Friedman, J. H. (2001). Greedy function approximation: a gradient boosting machine. *Annals of statistics*, 1189–1232.
- Ghani, R., Probst, K., Liu, Y., Krema, M., & Fano, A. (2006). Text mining for product attribute extraction. *ACM SIGKDD Explorations Newsletter*, 8(1), 41–48.
- Gupta, V., & Murzova, A. (2017). *Keras tutorial : Using pre-trained imagenet models*. Retrieved 2021-11-19, from <https://learnopencv.com/keras-tutorial-using-pre-trained-imagenet-models/>
- Hang, H., Huang, T., Cai, Y., Yang, H., & Lin, Z. (2021). Gradient boosted binary histogram ensemble for large-scale regression. *arXiv preprint arXiv:2106.01986*.
- Harwood, B., Kumar BG, V., Carneiro, G., Reid, I., & Drummond, T. (2017). Smart mining for deep metric learning. In *Proceedings of the ieee international conference on computer vision* (pp. 2821–2829).
- Hess, A. (2018). Exploring feature reuse in densenet architectures. *arXiv preprint arXiv:1806.01935*.
- Huang, G., Liu, Z., Van Der Maaten, L., & Weinberger, K. Q. (2017). Densely connected convolutional networks. In *Proceedings of the ieee conference on computer vision and pattern recognition* (pp. 4700–4708).
- Ke, G., Meng, Q., Finley, T., Wang, T., Chen, W., Ma, W., ... Liu, T.-Y. (2017). Lightgbm: A highly efficient gradient boosting decision tree. *Advances in neural information processing systems*, 30, 3146–3154.
- Kingma, D. P., & Ba, J. (2014). Adam: A method for stochastic optimization. *arXiv preprint arXiv:1412.6980*.
- Ko, E. S. (2021). *Product matching through multimodal image and text combined similarity matching*.
- Köpcke, H., Thor, A., & Rahm, E. (2010). Evaluation of entity resolution

- approaches on real-world match problems. *Proceedings of the VLDB Endowment*, 3(1-2), 484–493.
- Li, J., Dou, Z., Zhu, Y., Zuo, X., & Wen, J.-R. (2020). Deep cross-platform product matching in e-commerce. *Information Retrieval Journal*, 23(2), 136–158.
- Miller, S. J., Howard, J., Adams, P., Schwan, M., & Slater, R. (2020). Multi-modal classification using images and text. *SMU Data Science Review*, 3(3), 6.
- Moindrot, O. (2018). *Triplet loss and online triplet mining in tensorflow*. Retrieved 2021-11-20, from <https://omoindrot.github.io/triplet-loss#why-not-just-use-softmax>
- Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., Thirion, B., Grisel, O., ... Duchesnay, E. (2011). Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research*, 12, 2825–2830.
- Ristoski, P., Petrovski, P., Mika, P., & Paulheim, H. (2018). A machine learning approach for product matching and categorization. *Semantic web*, 9(5), 707–728.
- Schroff, F., Kalenichenko, D., & Philbin, J. (2015). Facenet: A unified embedding for face recognition and clustering. In *Proceedings of the IEEE conference on computer vision and pattern recognition* (pp. 815–823).
- Shah, K., Kopru, S., & Ruvini, J. D. (2018). Neural network based extreme classification and similarity models for product matching. In *Proceedings of the 2018 conference of the north american chapter of the association for computational linguistics: Human language technologies, volume 3 (industry papers)* (pp. 8–15).
- Shorten, C., & Khoshgoftaar, T. M. (2019). A survey on image data augmentation for deep learning. *Journal of Big Data*, 6(1), 1–48.
- Speer, R. (2019). *ftfy*. Zenodo. Retrieved from <https://doi.org/10.5281/zenodo.2591652> (Version 5.5) doi: 10.5281/zenodo.2591652
- Tajbakhsh, N., Shin, J. Y., Gurudu, S. R., Hurst, R. T., Kendall, C. B., Gotway, M. B., & Liang, J. (2016). Convolutional neural networks for medical image analysis: Full training or fine tuning? *IEEE transactions on medical imaging*, 35(5), 1299–1312.
- Tracz, J., Wójcik, P. I., Jasinska-Kobus, K., Belluzzo, R., Mroczkowski, R., & Gawlik, I. (2020). Bert-based similarity learning for product matching. In *Proceedings of workshop on natural language processing in e-commerce* (pp. 66–75).
- Van Rossum, G., & Drake, F. L. (2009). *Python 3 reference manual*. Scotts Valley, CA: CreateSpace.
- Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., ... Polosukhin, I. (2017). Attention is all you need. In *Advances in neural information processing systems* (pp. 5998–6008).

- Wang, Z., Liang, X., & Wang, C. (2021). Controllable text-to-image generation with enhanced text encoder and edge-preserving embedding. In *Journal of physics: Conference series* (Vol. 1856, p. 012003).
- Wei, Y., Tran, S., Xu, S., Kang, B., & Springer, M. (2020). Deep learning for retail product recognition: Challenges and techniques. *Computational intelligence and neuroscience*, 2020.
- Wilke, M., & Rahm, E. (2021). Towards multi-modal entity resolution for product matching.
- Wolf, T., Debut, L., Sanh, V., Chaumond, J., Delangue, C., Moi, A., ... others (2019). Huggingface's transformers: State-of-the-art natural language processing. *arXiv preprint arXiv:1910.03771*.
- Yang, Y., Cer, D., Ahmad, A., Guo, M., Law, J., Constant, N., ... others (2019). Multilingual universal sentence encoder for semantic retrieval. *arXiv preprint arXiv:1907.04307*.
- Zaharia, M., Xin, R. S., Wendell, P., Das, T., Armbrust, M., Dave, A., ... others (2016). Apache spark: a unified engine for big data processing. *Communications of the ACM*, 59(11), 56–65.

APPENDIX A: FEATURES IN RAW DATASETS

Table 4: Features in the original Jumbo active product dataset

Feature	Type	Description
jumbo_id	string	Unique identifier of Jumbo product
title	string	Product title
brand_desc	string	Name of brand
uom_volume	string	Unit of measure
volume	float	Number of units of measure contained in the product
topLevelCategory	string	Main product category to which the product belong
topLevelCategoryId	string	Unique identifier of the main product category
jumbo_img_1	array of strings	Path(s) to image(s) of the primary view of the product
jumbo_img_2	array of strings	Path(s) to image(s) of the other views of the product

Table 5: Features in the original competitor product dataset

Feature	Type	Description
comp_article_ext_id	string	Identifier of competitor product
comp_banner_id	integer	Identifier of competitor
comp_article_id	integer	Internal reference to competitor product
comp_banner_name	string	Name of competitor
door_id	integer	Another identifier of competitor
comp_article_desc	string	Product title
comp_catg_desc	string	Product category to which the product belong
comp_brand_name	string	Name of brand
jumbo_id	string	Unique identifier of Jumbo counterpart with which the product is matched
comparable_quality	binary	Whether the Jumbo counterpart is exactly the same as the competitor product
conversion_quality	float	Ratio of volume of Jumbo counterpart to that of the competitor product
comp_article_size	string	Product size
comp_article_uom	string	Unit of measure
comp_article_volume	string	Number of units of measure contained in the product
comp_article_status	string	Whether the competitor product is matched (must be true) and whether there is size/brand mismatch
img_dir	string	Path to image of the product

APPENDIX B: RESULTS OF CLASSIFICATION MODELS

Table 6: Comparison of test prediction performance between binary and multi-label classification models

		Exact Match Ratio	Micro-averaged Precision	Micro-averaged Recall
Binary	Overall	36.90%	40.55%	80.91%
	Branded	37.75%	39.88%	89.47%
	Private Label	34.67%	43.78%	57.20%
Multi-label	Overall	65.61%	70.17%	88.43%
	Branded	71.39%	75.29%	92.95%
	Private Label	50.56%	57.04%	75.89%
Multi-label w/ Size	Overall	69.64%	75.12%	89.45%
	Branded	74.55%	78.74%	93.60%
	Private Label	56.87%	65.17%	77.95%
Multi-label w/ PL Indicator	Overall	65.82%	70.62%	88.27%
	Branded	72.79%	77.36%	91.97%
	Private Label	47.71%	54.98%	78.01%
Multi-label w/ All Extra Features	Overall	70.57%	75.67%	89.63%
	Branded	76.16%	80.72%	92.74%
	Private Label	56.06%	63.18%	81.04%

SOURCE CODE

The code for the experiments in this research can be found at <https://github.com/ulyko1996/Master-Thesis>. It is only accessible to reviewers of this thesis.